

MIP_REPOSITORY_STRUCTURE.md

Mainframe Intelligence Platform

Repository & Engineering Structure

Version: 1.0

Purpose

Define the repository structure, package boundaries, service architecture, plugin architecture, deployment model, and development standards for MIP.

This document serves as the implementation blueprint for engineering teams.

Repository Strategy

Recommendation

Use a Monorepo.

Reason:

MIP components are highly coupled during the early phases.

Examples:

- Parsers depend on Domain Models
- Graph Builder depends on Metadata
- Reasoning depends on Graph
- AI depends on Reasoning

A monorepo simplifies:

- Development
 - Refactoring
 - CI/CD
 - Shared models
 - Shared testing
-

Repository Structure

```
mip-platform/  
├─ mfcode/  
├─ docs/  
├─ infrastructure/  
├─ services/  
├─ packages/  
├─ skills/  
├─ prompts/  
├─ tests/  
├─ scripts/  
├─ deployments/  
├─ .github/  
└─ README.md
```

Source Repository Folder

```
mfcode/
```

Purpose:

Read-only source artifacts.

Examples:

```
mfcode/  
├─ cobol/  
├─ jcl/  
├─ copybooks/  
└─ db2/
```

```
|— cics/  
|— scheduler/  
└— docs/
```

Never modify source assets.

Packages Layer

Contains reusable libraries.

```
packages/  
  
|— domain/  
|— metadata/  
|— graph/  
|— parsing/  
|— reasoning/  
|— ai/  
└— common/
```

Package: domain

Purpose:

Canonical domain model.

Contains:

```
packages/domain/  
  
|— entities/  
|— relationships/  
|— enums/  
└— schemas/
```

Examples:

- Program
 - Job
 - Copybook
 - Table
 - Dataset
-

Package: parsing

Purpose:

Parsing framework.

```
packages/parsing/
```

```
|— core/  
|— plugins/  
|— contracts/
```

Parser Plugin Architecture

Every parser implements:

```
ParserPlugin
```

Contract:

```
discover()  
  
parse()  
  
validate()  
  
extract_metadata()
```

Supported Plugins:

```
COBOL  
JCL  
DB2  
VSAM  
IMS  
CICS  
Scheduler
```

Future:

```
Java  
Spring Boot
```

.NET
Kafka
REST APIs

Package: metadata

Purpose:

Metadata repository abstraction.

Contains:

```
repositories/  
services/  
models/
```

Supported stores:

- PostgreSQL
 - SQLite (local)
 - Future cloud databases
-

Package: graph

Purpose:

Knowledge graph services.

Contains:

```
nodes/  
edges/  
queries/  
loaders/  
services/
```

Supported Graph Engines:

- Neo4j
- NetworkX

Future:

- Neptune
 - Cosmos Graph
-

Package: reasoning

Purpose:

Enterprise intelligence.

Capabilities:

```
Impact Analysis
Root Discovery
Dependency Analysis
Flow Analysis
Business Rule Discovery
```

Package: ai

Purpose:

AI orchestration.

Contains:

```
agents/
tools/
prompts/
retrievers/
```

Framework:

- LangGraph
-

Services Layer

Microservices exposed through APIs.

```
services/
```

```
|— discovery-service/  
|— metadata-service/  
|— graph-service/  
|— reasoning-service/  
|— copilot-service/  
└— modernization-service/
```

Discovery Service

Responsibilities:

- Repository scanning
- Artifact classification
- Change detection

Outputs:

Metadata events

Metadata Service

Responsibilities:

- Metadata persistence
 - Metadata retrieval
 - Metadata validation
-

Graph Service

Responsibilities:

- Graph construction
 - Graph updates
 - Graph queries
-

Reasoning Service

Responsibilities:

- Dependency tracing
 - Impact analysis
 - Root discovery
-

Copilot Service

Responsibilities:

- Natural language interaction
 - Context assembly
 - Response generation
-

Modernization Service

Responsibilities:

- Service discovery
 - API discovery
 - Event discovery
 - Migration recommendations
-

Skills Structure

```
skills/  
  
├─ repository-discovery/  
├─ cobol-analysis/  
├─ jcl-analysis/  
├─ db2-lineage/  
├─ graph-builder/  
├─ impact-analysis/  
├─ business-rules/  
└─ modernization-advisor/
```

Each skill contains:


```
skill.md
examples/
templates/
tests/
```

Prompt Structure

```
prompts/

├─ architecture/
├─ implementation/
├─ review/
├─ testing/
├─ graph/
├─ reasoning/
└─ modernization/
```

Purpose:

Standardize Copilot and AI outputs.

Test Structure

```
tests/

├─ unit/
├─ integration/
├─ graph/
├─ reasoning/
└─ e2e/
```

CI/CD Structure

```
.github/

├─ workflows/
├─
├─ pull_request.yml
└─ test.yml
```

```
|— lint.yml
|— release.yml
```

Development Workflow

Step 1

Architecture Design

Step 2

Architecture Review

Step 3

Implementation

Step 4

Testing

Step 5

Code Review

Step 6

Documentation

Step 7

Release

Deployment Architecture

Local Development

```
VS Code
Docker
PostgreSQL
Neo4j
```

Enterprise Deployment

Kubernetes

- |— Discovery Service
- |— Metadata Service
- |— Graph Service
- |— Reasoning Service
- |— Copilot Service

Supporting Components:

PostgreSQL
Neo4j
Redis
Kafka
OpenSearch

Observability

Mandatory:

- OpenTelemetry
- Prometheus
- Grafana

Metrics:

- Files processed
- Metadata extracted
- Graph nodes created
- Graph edges created
- Query latency
- AI response latency

Security Model

Support:

- RBAC
- Audit Trails
- Encryption
- Repository Access Controls
- Source Code Governance

MVP Repository Scope

Build only:

Repository Discovery
COBOL Parser
JCL Parser
Metadata Repository
Knowledge Graph
Dependency Queries

Do NOT build:

AI Copilot
Modernization Engine
Business Rule Extraction

These come later.

Engineering Rule

Every new capability must answer:

1. What metadata does it create?
2. What graph entities does it create?
3. What relationships does it create?
4. What questions can it answer?

If those questions cannot be answered, the capability should not be implemented.

End State

MIP becomes a reusable enterprise platform where:

Source Systems ↓ Metadata ↓ Knowledge Graph ↓ Reasoning ↓ AI ↓ Modernization

All operate on a shared canonical model and architecture.